

Dataflow, Graphs, and Parallel Symmetries

This chapter examines how streaming computation, graph structure, and symmetry interact in parallel systems. The central concern is not only whether a computation can be expressed as a dataflow graph, but whether that graph admits compositional scheduling, symmetry-aware optimization, and implementation choices that preserve throughput under resource constraints. In this setting, performance is summarized by an error-and-efficiency profile such as $\varepsilon(r)$, where the relevant observables include throughput, latency, and deadline miss rate.

A useful starting point is the class of Kahn networks, in which processes communicate through unbounded FIFO channels and the overall behavior is determined compositionally by the local process semantics. Such networks provide a clean model for streaming systems because they separate functional behavior from scheduling policy. In practice, however, the semantics of a streaming pipeline must be paired with a compositional scheduling discipline: the order in which actors fire, the allocation of buffers, and the handling of backpressure all affect whether the implementation meets its timing targets. Streaming operads offer one way to formalize this compositionality, treating pipeline fragments as operations that can be assembled while preserving typing and interface structure.

Graph structure becomes especially important when the pipeline exhibits nontrivial automorphisms. If a dataflow graph has symmetries, then equivalent subcomputations may be permuted without changing the observable behavior. These graph automorphisms can be exploited to obtain load-balanced symmetries: work can be distributed across processors or accelerators in a way that respects the graph’s invariant structure while reducing imbalance. In favorable cases, symmetry-aware scheduling identifies interchangeable regions of the graph and uses them to improve utilization without changing the denotation of the computation.

The optimization laws in this chapter are expressed through fusion and fission. Fusion combines adjacent stages to reduce communication overhead and improve locality, while fission splits a stage into parallel copies to increase throughput or match hardware granularity. These transformations are not merely heuristic; they are constrained by the semantics of the dataflow graph, by buffer capacities, and by the need to preserve the relevant performance metrics. On GPUs and FPGAs, such laws often appear as tiling decisions: a streaming kernel may be partitioned into tiles that fit shared memory, register files, or on-chip resources, and the resulting tiling must be compatible with the graph’s dependency structure.

A compact way to state the chapter’s design principle is that symmetry should be treated as an optimization resource. When a pipeline admits automorphisms, those symmetries can guide both scheduling and placement, yielding implementations that are not only correct but also balanced across available hardware. Conversely, when fusion or fission breaks an apparent symmetry, the resulting asymmetry must be justified by a measurable gain in $\varepsilon(r)$, such as improved throughput, reduced latency, or a lower deadline miss rate.

The artifact associated with this chapter is a family of streaming kernels equipped with symmetry transforms. The intended use is to demonstrate how a kernel can be rewritten by graph automorphisms, fused or fissioned according to resource constraints, and then mapped to a parallel target such as a GPU or FPGA. The artifact should record the original graph, the transformed graph, the scheduling assumptions, and the measured values of throughput, latency, and deadline miss rate so that the optimization can be reproduced and compared across implementations.

Taken together, these ideas support a symmetry-aware view of parallel optimization: the graph provides the compositional skeleton, the scheduling discipline determines executable behavior, and the symmetry structure identifies legal transformations that can improve performance. The chapter’s guiding principle is that parallel efficiency is not an external afterthought but a property

that can be tracked, transformed, and validated within the same compositional framework as the computation itself.